Apuntes del Sprint 10 · Redes neuronales: cómo funcionan, cómo se entrenan y cuándo valen la pena

ANTES DE EMPEZAR
este sprint reutiliza ideas del Sprint 5 (descenso del gradiente), del Sprint 7 (regresión logística, sigmoide) y del Sprint 8 (el campeón con el que comparar). Si alguna te suena a chino, repasa esos apuntes.

Las diapositivas del Sprint 10 muestran mucho código de Keras en poco espacio. Estos apuntes **explican qué hace una red por dentro, con cuentas pequeñas que puedes hacer a mano**. Si entiendes estas cuentas, entenderás qué hace Keras cuando escribes `fit`.

0. De qué va el sprint (en 1 minuto)

La dirección de TurisData **oye hablar de redes neuronales en todas partes** y pregunta si debe sustituir su modelo de cancelaciones.

> **La pregunta del reto:** *¿Una red neuronal mejora al aprendizaje automático clásico, y a qué precio?* Quiere **evidencias con el coste incluido**, no una opinión. **Un resultado honesto de «no compensa» es tan válido como un «sí».**

Presentación	Pregunta	Cuaderno
S10_01 Red neuronal desde cero	¿Qué es una red por dentro?	S10_01 (solo NumPy)
S10_02 Redes densas con Keras	¿Cómo la construyo y la aplico a TurisData?	S10_02
S10_03 Entrenamiento y regularización	¿Cómo evito que memorice?	S10_03, S10_04 (reto)

PARTE 1 · Qué es una red neuronal por dentro

1. La neurona: tres pasos

Una **neurona artificial** hace **tres cosas**:

1. **Multiplica** cada entrada por un **peso** (cuánto le importa).
2. **Suma** todo y añade un **sesgo** (*bias*).
3. Pasa el resultado por una **función de activación** (un «interruptor suave»).

```
def neurona(x, pesos, sesgo):
    return sigmoide(np.dot(x, pesos) + sesgo)
```

Ejemplo con números: entradas `x = (2, 1)`, pesos `(0,5, −1)`, sesgo `0,5`.

```
Suma ponderada:  z = 2·0,5 + 1·(−1) + 0,5 = 1 − 1 + 0,5 = 0,5
Activación sigmoide:  1 / (1 + e^(−0,5)) = 0,62
```

Es la misma suma ponderada de la regresión logística (Sprint 7): una red es muchas de estas neuronas apiladas.

2. Funciones de activación

> **Sin activación, apilar capas no sirve de nada:** una suma de sumas ponderadas sigue siendo **un modelo lineal**. La activación introduce **curvas**.

Función	Salida	Uso típico
Sigmoide	Entre 0 y 1	**Capa final** de clasificación binaria
ReLU	**0 si es negativo; z si es positivo**	**Capas ocultas**

```
def relu(z): return np.maximum(0, z)
def relu_derivada(z): return (z > 0).astype(float)
```

ReLU de −2 → **0**; de 0 → **0**; de 3 → **3**. Es la más usada porque es **muy simple y rápida**.

> **Una neurona ReLU «apagada» (z ≤ 0) no aprende en esa muestra**, porque su derivada es 0.

3. Un problema que una línea no resuelve

Imagina **dos «anillos» de puntos**, uno dentro del otro (clientes de dos tipos que **no se separan con una recta**). La regresión logística **se queda en el azar**. Una red con **una capa oculta** sí puede **curvar** la frontera.

La red del cuaderno: 2 entradas → **capa oculta (ReLU)** → 1 salida (sigmoide).

Una **capa** = **muchas neuronas a la vez** = **una multiplicación de matrices**:

```
Z1 = X · W1 + b1      →     A1 = relu(Z1)      →     Z2 = A1 · W2 + b2      →     A2 = sigmoide(Z2)
```

Un pase hacia delante, con números

Entrada `X = (1, 2)`. Capa oculta con 2 neuronas, `W1 = [[1, −1], [0,5, 1]]`, `b1 = (0, −1)`. Salida con `W2 = (1, −0,5)`, `b2 = 0,1`.

Paso	Cálculo	Resultado
Neurona oculta 1	1·1 + 2·0,5 + 0	**2**
Neurona oculta 2	1·(−1) + 2·1 − 1	**0**
Activación ReLU	(2, 0)	(2, 0)
Salida (suma)	2·1 + 0·(−0,5) + 0,1	**2,1**
Sigmoide	1/(1 + e^(−2,1))	**0,89**

La red dice: **«89 % de probabilidad de la clase 1».** Esto es **propagación hacia delante** (*forward*): los datos entran por un lado y pasan, capa a capa, hasta la salida.

4. Aprender = descenso del gradiente (otra vez)

Los **pesos** empiezan siendo números al azar. **Aprender** es ajustarlos para que el error baje. Es **exactamente el bucle del Sprint 5**, con más pesos:

```
adelante  →  calcular el error  →  atrás (gradientes por retropropagación)  →  actualizar los pesos
```

Cada vuelta completa a los datos es una **época**.

```
def paso_gradiente(parametros, gradientes, tasa):
    return {k: parametros[k] - tasa * gradientes[k] for k in ("W1", "b1", "W2", "b2")}
```

La retropropagación (*backpropagation*) solo es **la regla de la cadena** del cálculo, aplicada de la salida hacia atrás para saber **cuánto «culpa» tiene cada peso del error**. Tú no la derivas a mano; el ordenador sí.

Ejemplo mínimo de un paso

Un solo peso: `w = 2`, entrada `x = 3`, valor real `y = 10`. La predicción es 2·3 = **6**; el error al cuadrado es (6 − 10)² = 16.

```
gradiente = 2·(w·x − y)·x = 2·(6 − 10)·3 = −24      (negativo: hay que SUBIR w)
w nuevo = w − tasa·gradiente = 2 − 0,01·(−24) = 2,24
nueva predicción = 2,24·3 = 6,72                     (más cerca de 10)
```

Repitiéndolo, w se acerca a 3,33 (porque 3,33·3 = 10). **Eso es entrenar.**

5. ¿Cuántas neuronas hacen falta?

El cuaderno entrena redes con **1, 2 y 8 neuronas ocultas** sobre los anillos: **con 1 no separa; con 8 sí.** Más neuronas = más capacidad para curvar la frontera.

> **Mini-reto:** una red de 4 neuronas con **tasa 0,01** durante 1.500 épocas **(`red_lenta`)**: *¿por qué le cuesta tanto?* Porque los pasos

son diminutos (Sprint 5: tasa pequeña = lento). *¿Y con una tasa demasiado alta?* La pérdida **salta y no converge**.

PARTE 2 · Redes densas con Keras

6. Preparar los datos para una red

Las redes son sensibles a la escala: si `antelacion_dias` va de 0 a 400 y `adultos` de 1 a 4, **la primera «grita» más**.

- **Numéricas:** estandarizar. **Categóricas:** one-hot.
- **Sin** `id_reserva`, **sin fechas** y **sin el objetivo** (la columna que se predice).

```
cols_num = ["antelacion_dias", "noches", "adultos", "ninos", "peticiones_especiales",
            "cliente_repetidor", "cancelaciones_previas", "mes_llegada"]
cols_cat = ["tipo_habitacion", "regimen", "canal", "pais_origen", "tarifa", "deposito"]
```

7. Un MLP para clasificar cancelaciones

MLP = *perceptrón multicapa*: una **pila de capas `Dense`**, donde **todas las neuronas de una capa se conectan con todas las de la siguiente**.

```
def construir_mlp_clasificacion(n_entradas, capas=(32, 16)):
    m = keras.Sequential([keras.layers.Input((n_entradas,))])
    for u in capas:
        m.add(keras.layers.Dense(u, activation="relu"))
    m.add(keras.layers.Dense(1, activation="sigmoid"))
    m.compile(optimizer="adam", loss="binary_crossentropy",
              metrics=[keras.metrics.AUC(name="auc")])
    return m
```

Cómo leerlo:

Pieza	Qué hace
`Dense(32, "relu")`, `Dense(16, "relu")`	Dos capas ocultas de 32 y 16 neuronas
`Dense(1, "sigmoid")`	**Una** neurona de salida: la **probabilidad de cancelar**
`optimizer="adam"`	La variante de descenso del gradiente que se usa casi siempre
`loss="binary_crossentropy"`	La función de error para clasificación 0/1

Cuántos parámetros tiene

Con 30 entradas y capas (32, 16):

```
Capa 1:  30·32 + 32 = 992          (30 entradas × 32 neuronas + 32 sesgos)
Capa 2:  32·16 + 16 = 528
Salida:  16·1 + 1   = 17
Total = 1.537 parámetros
```

Con capas (128, 64): **12.289** parámetros (ocho veces más). Si solo tienes 800 reservas de entrenamiento, **hay muchos más parámetros que datos: memorizar es fácil.**

La pérdida *binary_crossentropy*

Castiga mucho **estar muy seguro y equivocarse**:

Real	Predicción de la red	Pérdida
Cancela (1)	0,90	−ln(0,9) = **0,11** (bien)
Cancela (1)	0,50	**0,69** (dudoso)
Cancela (1)	0,10	**2,30** (muy mal)

8. Entrenar y comparar con la logística

```
hist = modelo.fit(A_ent, y_ent, epochs=15, batch_size=64, validation_data=(A_val, y_val))
auc_mlp = roc_auc_score(yc_pru, prob_mlp)
auc_log = roc_auc_score(yc_pru, prob_log)
mlp_gana = auc_mlp > auc_log + 0.02
```

- **Época:** una pasada por **todos los datos**.
- **Lote (`batch_size`):** cuántas filas se procesan **antes de actualizar los pesos**. Con 800 reservas y lote 64: 800/64 = 12,5 → **13 actualizaciones por época**; en 15 épocas, **195 actualizaciones**.
- **`validation_data`:** datos que la red **no usa para aprender**, solo para vigilar el sobreajuste.

> **¿El MLP gana por más de 0,02 de AUC?** Esa respuesta es **la pregunta del reto en pequeño**. **Con datos tabulares, no siempre.** Muchas veces el boosting (Sprint 8) iguala o supera a una red con una fracción del coste.

9. Un MLP para predecir el precio (regresión)

	Clasificación	**Regresión**
Capa de salida	`Dense(1, "sigmoid")`	`Dense(1)` **sin activación**
Pérdida	`binary_crossentropy`	`mse`
Métrica legible	AUC	**MAE (euros)**

```
mae_mlp_euros = mean_absolute_error(yr_pru, pred_mlp)
mejora_vs_media = 1 - mae_mlp_euros / mae_base
```

Ejemplo: si predecir siempre la media da un MAE de 20 € (`mae_base`) y la red da 15 €, la mejora es 1 − 15/20 = **25 %**.

> El objetivo (`precio_noche`) **se escala para entrenar** y la predicción **se devuelve a euros** al final.

10. Mini-reto: ¿es mejor la red grande?

```
hist_ancho = construir_mlp_clasificacion(n, capas=(128, 64)).fit(A_ent, y_ent, epochs=15, batch_size=64, validation_data=
(A_val, y_val))
auc_ancho = hist_ancho.history["val_auc"][-1]
```

Más neuronas = más capacidad… y más facilidad para memorizar. Compara `auc_ancho` con el MLP pequeño y escribe tu conclusión. Esto lleva a la siguiente parte: **regularización**.

PARTE 3 · Entrenamiento y regularización

11. Diagnosticar el sobreajuste

En el cuaderno S10_03 entrenas **una red grande con solo 800 reservas**: memorizar es facilísimo. **Las curvas lo cuentan:**

```
Pérdida de ENTRENAMIENTO:  baja, baja, baja…                  (sigue bajando siempre)
Pérdida de VALIDACIÓN:     baja, toca un mínimo y VUELVE A SUBIR  ← sobreajuste
```

Época	**Pérdida entrenamiento**	**Pérdida validación**
5	0,60	0,62
12	0,45	**0,55** ← mínimo
30	0,20	0,80
60	0,05	1,10

(Valores ilustrativos.)

```
epoca_optima = int(np.argmin(h_base.history["val_loss"])) + 1
brecha_final = h_base.history["val_loss"][-1] - h_base.history["loss"][-1]
```

En el ejemplo, `epoca_optima` = 12 y se entrenó 60: **si `epoca_optima` es muy inferior al nº de épocas, has entrenado de más.** La brecha final es 1,10 − 0,05 = **1,05**.

12. Las herramientas de regularización

Técnica	Idea en una frase	En Keras
L2 (*weight decay*)	**Castiga los pesos grandes**	`kernel_regularizer=keras.regularizers.l2(0.01)`
Dropout	**Apaga al azar un % de neuronas** en cada paso	`keras.layers.Dropout(0.4)`
Parada temprana	**Detiene** cuando la validación deja de mejorar	`EarlyStopping(patience=8, restore_best_weights=True)`
Reducir la tasa	**Pasos más pequeños** al estancarse	`ReduceLROnPlateau(factor=0.5, patience=4)`

Entendiéndolas con ejemplos:

- **L2:** se suma al error una multa por los pesos grandes. Con `l2 = 0,01` y pesos (3, −4): multa = 0,01·(9 + 16) = **0,25**. Así la red prefiere varios pesos moderados a unos pocos enormes (mismo efecto que Ridge del Sprint 7).
- **Dropout 0,4:** en **cada paso de entrenamiento**, el 40 % de las neuronas «se apagan» al azar. Así **ninguna neurona puede depender de otra concreta** y la red no memoriza con facilidad. **Es como estudiar con distintos compañeros ausentes cada día.** En la predicción final **todas las neuronas están activas**.
- **Parada temprana (`patience = 8`):** si pasan 8 épocas sin que la validación mejore, **para** y `restore_best_weights=True` **recupera los pesos de la mejor época** (la 12 del ejemplo, no la 60).
- **`ReduceLROnPlateau`:** si la validación se estanca 4 épocas, **la tasa se reduce a la mitad** (para afinar).

13. Callbacks, tasa de aprendizaje y lote

```
mis_callbacks = [EarlyStopping(monitor="val_loss", patience=8, restore_best_weights=True),
                 ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=4)]
m_es, h_es = entrenar(A_peq, y_peq, epocas=100, callbacks=mis_callbacks, l2=0.005, dropout=0.3)
```

Un **callback** es una **función que Keras llama automáticamente en cada época** (para parar, guardar, ajustar la tasa…). Aquí se combinan las cuatro herramientas.

Ajuste	Demasiado alto	Demasiado bajo
Tasa de aprendizaje	La pérdida **salta y no converge**	**Tarda una eternidad**
Lote	Más estable, **menos actualizaciones** (lote grande)	Más actualizaciones pero **más ruido** (lote pequeño)

14. Guardar y cargar el modelo

```
m_es.save(ruta_modelo)                          # formato .keras
m_cargado = keras.models.load_model(ruta_modelo)
prob_cargado = m_cargado.predict(A_val)         # idéntico al original
```

Un modelo solo sirve si se puede reutilizar sin reentrenar. Y en producción hay que aplicar **exactamente el mismo preprocesado** (el `pre`): lo ideal es **guardarlo junto al modelo**. Si el escalador cambia, las predicciones carecen de sentido.

15. El reto: ¿vale la pena la red?

Cuatro reglas para que la comparación sea **justa**:

Paso	Qué hacer
1. Misma partición	**64 % entrenamiento, 16 % validación** (decisiones y curvas), **20 % prueba** (**una sola vez**)
2. Referencia clásica	El boosting del cuaderno o **tu campeón del Sprint 8**, **midiendo también el tiempo**
3. MLP con buenas prácticas	**Pequeño, con L2, dropout y parada temprana**, entrenado con **3 semillas**
4. Veredicto	Media de las 3 semillas; **intervalo de confianza de la diferencia**

¿Por qué 3 semillas? Una red depende del **azar de los pesos iniciales**. Con una sola semilla podrías haber tenido suerte. Con 3 ves **cuánto varía**.

Cómo decidir «hay diferencia»

Se calcula el **intervalo de confianza de la diferencia** de AUC entre MLP y boosting (`inf`, `sup`):

Intervalo	Veredicto
(+0,01 ; +0,04) — **todo por encima de 0**	**Gana el MLP**
(−0,03 ; −0,01) — **todo por debajo de 0**	**Gana el boosting**
(−0,02 ; +0,02) — **incluye el 0**	**Empate** (no hay evidencia de diferencia)

`hay_diferencia = inf > 0 or sup < 0`. Y el campo `mejor_modelo` solo puede valer `"MLP"`, `"Boosting"` o `"Empate"`.

> **El informe de una página** tiene: la **pregunta y su respuesta en una frase**, una **tabla con incertidumbre y coste** (tiempo, complejidad) y una **recomendación**. *«Con una diferencia de AUC de 0,005 (IC: −0,01 a +0,02) y 40 veces más tiempo de entrenamiento, no compensa sustituir el boosting»* es una conclusión excelente.

Las partes marcadas **«en casa»** (tasa de aprendizaje, guardar el modelo y mini-retos) **no son necesarias para el reto**.

---

16. Dónde practicar cada cosa

Si no entiendes...	Mira la sección	Practica en
Neurona, activación, capas, retropropagación	1-5	`S10_01_red_desde_cero`
Preparar datos, MLP en Keras, pérdida, parámetros	6-10	`S10_02_mlp_keras`
Sobreajuste, L2, dropout, parada temprana, callbacks	11-14	`S10_03_regularizacion`
Comparación justa y veredicto	15	`S10_04_reto_vale_la_pena`

17. Chuleta de vocabulario

Palabra	En lengua normal
Neurona	Multiplica, suma, aplica una activación
Peso / sesgo	Cuánto importa cada entrada / un «empujón» fijo
Activación	«Interruptor suave» que da curvas a la red
ReLU / sigmoide	0 si negativo / comprime entre 0 y 1
Capa densa	Todas las neuronas conectadas con la siguiente capa
MLP	Pila de capas densas
Propagación hacia delante	De los datos a la predicción
Retropropagación	Calcular cuánto «culpa» tiene cada peso del error
Época	Una pasada por todos los datos
Lote (batch)	Filas procesadas antes de actualizar los pesos
Pérdida (loss)	El número que la red intenta reducir
Adam	Variante del descenso del gradiente muy usada
L2 / weight decay	Multa a los pesos grandes
Dropout	Apagar neuronas al azar al entrenar
Parada temprana	Parar cuando la validación deja de mejorar
Callback	Función que se ejecuta automáticamente en cada época

Palabra	En lengua normal
Semilla	Número que fija el azar para repetir resultados

18. Cómo estudiar estos apuntes

1. **Antes de cada cuaderno**, lee la sección que indica la tabla del apartado 16.
2. **Repite a mano** la neurona (sección 1) y el pase hacia delante (sección 3): son 5 multiplicaciones y te quitan el miedo a las redes.
3. **Aprende a leer las curvas**: la clave del sprint es ver **cuándo la validación se separa del entrenamiento**.
4. **En el reto, no defiendas la red por ser «más moderna»**: defiende con **evidencias y coste**. Un honesto «no compensa» es una buena respuesta.